

Lab 3: Architectural Specification

Student: Sujay Hegde / SRN: PES1UG24CS478

System: Self-Service Coffee Kiosk System

Selected style: Layered Architecture

1. Scenario and requirements

The busy cafe needs a touchscreen kiosk that lets customers select Espresso, Americano, or Latte in Small or Large sizes, pay by credit card, and receive a printed receipt. The system must store menu data and pricing and connect to receipt-printer hardware.

The design should keep touch interactions responsive, protect payment information, and handle declined payments and printer faults. Local storage, menu caching, a card-terminal SDK, and a USB/serial printer driver are proposed design choices. No measured response time or payment compliance certification is claimed.

2. Architectural style analysis

Style / Advantages for this scenario / Disadvantages for this scenario

Layered / Separates touch UI, order rules, and hardware/data adapters; simple deployment for one kiosk / Contract changes can affect multiple layers; one kiosk host remains a failure point

Microservices / Independent deployment and scaling across a large kiosk fleet / Adds network calls, service monitoring, and distributed transaction handling for a small application

Client-Server / Centralizes pricing and management across multiple kiosks / A remote backend adds connectivity dependency and possible server bottlenecks; caching can mitigate this

Layered Architecture fits the small, clearly divided workflow. These styles are not mutually exclusive: a layered kiosk can still contact an external payment server.

3. Architecture selection

We chose Layered Architecture for the Self-Service Coffee Kiosk System.

Reason 1 - Separation of concerns: Touchscreen screens change independently of order validation and pricing. UI uses Order Interface and does not access storage or printer drivers directly.

Reason 2 - Replaceable adapters: Payment, receipt printing, and menu storage each expose a stable contract. A different card terminal or printer can be supported by replacing its adapter without rewriting the order workflow.

Security advantage: A terminal SDK handles card capture and authorization. Order Manager receives an approval/decline result and transaction reference; storage and receipts contain no raw card data. This reduces unnecessary exposure. Logical component separation alone does not create process isolation or establish compliance.

Performance benefit: Local UI/order calls and cached menu prices avoid remote round trips for selection and total calculation. Payment authorization can still require a network call, and physical printing takes time. Actual latency requires measurement.

4. Components and logical layers

Component / Layer / Responsibility

User Interface / Presentation / Handle touch input; display coffee types, sizes, and payment/receipt status

Order Manager / Application / business / Validate selections, retrieve prices, manage order state, request payment, and print after approval

Payment Service / Infrastructure / Wrap card-terminal SDK; return approval, decline, or timeout and transaction reference

Receipt Printer / Infrastructure / Format order details, wrap printer hardware, report status, and support receipt retry

Menu Repository / Infrastructure / data / Store and read three coffee types, two sizes, and their prices in local storage

These are logical layers in one application, not separately deployed tiers. Order Manager requires three infrastructure contracts. UI depends on Order Manager; infrastructure components do not call UI.

5. Provided and required interfaces

Interface / Required by (socket) / Provided by (ball) / Operations / technology

Order Interface / User Interface / Order Manager / getMenu(), submitOrder(type, size), getStatus(orderId); local API calls

Payment Interface / Order Manager / Payment Service / processPayment(orderId, amount); local API wrapping card-terminal SDK

Receipt Interface / Order Manager / Receipt Printer / printReceipt(orderDetails), getPrinterStatus(); local API wrapping USB/serial driver

Menu Interface / Order Manager / Menu Repository / getMenu(), getPrice(type, size); repository API and database queries

A hollow circle denotes each provided interface; a semicircular socket denotes each required interface. Solid stems connect the consumer and provider through the assembled interface. Requests travel from the requiring component to the provider; results return through the same contract. Internal API labels are distinguished from hardware or storage technology behind an adapter.

6. Data flow and error handling

1. UI requests choices through Order Interface. Order Manager reads Menu Interface and returns the coffee types, sizes, and prices.
2. UI submits a selection. Order Manager validates it and calculates the amount from repository data.
3. Order Manager requests credit-card payment through Payment Interface. Payment Service returns the outcome and a transaction reference.
4. If approved, Order Manager calls Receipt Interface with the order details. UI displays the final order and printer status.
5. If declined, no paid receipt is printed. For an uncertain timeout, reconcile the original transaction before retrying payment to avoid duplicate charges.
6. If payment succeeds but printing fails, retain the paid order's receipt details for a print retry. Reprinting must not initiate a second payment.

7. Handout deliverable checklist

- Five components, including Order Manager and Payment Service.
- Four interfaces with explicit provider and consumer notation.
- Solid assembly connectors and labeled communication technologies.
- PNG and PDF diagram exports, plus editable draw.io and SVG sources.
- One-page Word justification and PDF with choice, two reasons, security advantage, and performance benefit.
- Comparison of Layered, Microservices, and Client-Server styles.